A0: Warm-up: the five tools

Assignment handout

Training dynamics, a self-study course · written by Claude for Fan Pu

October 29, 2026

about 4 hours · CPU only · every check runs in under a minute

1 Overview

Every later assignment treats training as a *dynamical system*: a state (the parameters) moving under a fixed rule (the optimizer). This warm-up asks: can you turn a training rule into a differential equation and read off its fixed points, their stability, its time scales and its noise? It drills the five prerequisite tools of the course brief (gradient flow, phase portraits, Hessians, eigendecompositions, stochastic differential equations) on objects small enough to do by hand, chiefly the *anchor model* of James Ang's post [J. M. L. Ang, 2026]: a two-layer scalar linear network $f_{a,b}(x) = abx$ trained on $y = x$ with

$$L(a, b) = \frac{1}{2}(ab - 1)^2, \tag{1}$$

after normalizing $\frac{1}{n}\sum_i x_i^2 = 1$ (harmless; the post's §2). Nothing comes before A0. A1 (conservation laws) uses the flow, the phase portrait and the mode decomposition; A3 (edge of stability) starts from the $2/\eta$ threshold and sharpness; A4 (SGD noise) from the Ornstein–Uhlenbeck process.

What you will derive

1. GD is forward Euler on gradient flow, with one-step error $O(\eta^2)$ (Problem (`flow-limit`))
2. When GD on a quadratic converges, oscillates or diverges, and how fast (Problem (`gd-stability`))
3. The fixed points of the anchor's gradient flow and their linear stability (Problem (`phase-portrait`))
4. The closed-form gradient flow of linear regression, mode by mode (Problem (`modes`))
5. The mean and variance of the Ornstein–Uhlenbeck process, in continuous and discrete time (Problem (`ou`))

What you will implement

1. `gd` in `prereqs/gd.py` (Problem (`gd-stability`))
2. `classify_fixed_point` in `prereqs/fixed_points.py` (Problem (`phase-portrait`))
3. `hvp`, `power_iteration`, `hutchinson_trace` in `prereqs/hessian.py` (Problem (`hessian`))
4. `mode_dynamics` in `prereqs/modes.py`, your closed form typed in (Problem (`modes`))
5. `ou_simulate` in `prereqs/sde.py` (Problem (`ou`))

What you will run

1. Six check scripts in `scripts/`, one per problem, each comparing a measurement with your derivation
2. A reading of the two framing papers, with three questions for `notes.md` (Problem (`framing`))

Setup. From the course root: `uv sync`, then `uv run pytest a0-prereqs` (all 48 tests fail with `NotImplementedError` until you fill in the stubs). Each check runs as `uv run python a0-prereqs/scripts/check_<name>.py` and writes PNGs and its section of `a0-prereqs/out/results.json`; `uv run python -m tdyn.results a0-prereqs/out/results.json a0-prereqs/solutions/results.json` shows where you differ from the reference. Worked solutions: `solutions/solutions.pdf`. The shared package `tdyn/` holds

everything not tagged `[you]`, including staff versions of `gd` and the Hessian tools for later assignments; write your own here.

Notation. Parameters $\theta \in \mathbb{R}^d$, loss L, step size η, step index k, continuous time t. For the anchor, $\theta = (a, b)$, residual $r = ab - 1$, imbalance $c = a^2 - b^2$. $\nabla^2 L$ is the Hessian, $\lambda_{\max}$ its largest eigenvalue. A dot is a time derivative, $\dot{\theta} = d\theta/dt$.

2 Gradient flow: the small-step limit

Gradient descent (GD) updates

$$\theta_{k+1} = \theta_k - \eta \nabla L(\theta_k). \tag{2}$$

An *ordinary differential equation* (ODE) $\dot{\theta} = F(\theta)$ specifies a velocity at every point; its solution from $\theta(0)$ is a curve $\theta(t)$. *Gradient flow* is the ODE

$$\dot{\theta} = -\nabla L(\theta). \tag{3}$$

The simplest numerical ODE solver is *forward Euler* with time step h: $\theta_{k+1} = \theta_k + hF(\theta_k)$, i.e. follow the current velocity for time h. Its *local error* is the error of one step started on the exact solution; its *global error* is the accumulated error at a fixed final time T; a method has *order* p if the global error is $O(h^p)$. Ang's post (§1) derives Equation 3 as the $\eta \to 0$ limit of Equation 2 [J. M. L. Ang, 2026]; this problem makes the limit quantitative and finds where it fails, which matters because most theory in the course is about the flow.

Problem (`flow-limit`): GD is forward Euler on gradient flow [Derive + Check + Predict, 25 min] **`[you]`**

(a) **`[Derive]`** Show that Equation 2 is forward Euler applied to Equation 3, and say which time t the iterate θ_k approximates. **Deliverable**: the time t_k as a function of k and η.

(b) **`[Derive]`** Expand the exact flow $\theta(\eta)$ started at θ_0 to second order in η, and compare it with one GD step θ_1. **Deliverable**: the leading term of $\theta(\eta) - \theta_1$ as a formula in $\nabla L(\theta_0)$ and $\nabla^2 L(\theta_0)$, the power of η it carries, and (by counting steps to reach a fixed time T) the power p in a global error of $O(\eta^p)$. *Hints, gentle to strong:* (1) $\theta(\eta) = \theta(0) + \eta\dot{\theta}(0) + \frac{\eta^2}{2}\ddot{\theta}(0) + O(\eta^3)$. (2) For $\ddot{\theta}$, differentiate $-\nabla L(\theta(t))$ in t by the chain rule: the derivative of ∇L along a curve is $\nabla^2 L$ times the velocity. (3) Reaching time T takes T/η steps; assume the per-step errors add without growing.

(c) **`[Check]`** Run `scripts/check_flow_limit.py`. It runs GD on the anchor Equation 1 from $(a, b) = (1.5, 0.5)$ at $\eta \in \{0.1, 0.03, 0.01, 0.003\}$, compares with an accurate solution of the flow at the same times, and fits log-log slopes of the one-step error and of the largest error on $t \in [0, 10]$ against η (`flow_limit_error.png`). The two slopes should match your two powers to within 0.1. A slope near 1 for the one-step error would mean your expansion missed a first-order term.

(d) **`[Predict]`** The script also runs $\eta = 0.9$ from the same start. Before opening `flow_limit_eta09.png`, write one line in `notes.md`: does GD at $\eta = 0.9$ end at the same minimum as the flow? Then look at the figure and write two sentences on why the small-step picture fails here even though the loss is the same smooth function. (Problem (`gd-stability`) and Problem (`hessian`) give you the vocabulary; come back to this.) **Deliverable**: the prediction, written before running, and the two sentences.

3 Stability of gradient descent on a quadratic

The single most useful fact in this course is when GD on a quadratic converges. Near a minimum θ^* any smooth loss looks like a quadratic, $L(\theta) \approx L(\theta^*) + \frac{1}{2}(\theta - \theta^*)^\top H(\theta - \theta^*)$ with $H = \nabla^2 L(\theta^*)$, so this is the local picture of GD near any minimum. Cohen et al. found that full-batch GD on neural networks drives $\lambda_{\max}(\nabla^2 L)$ up to the threshold this problem derives and then hovers there, the *edge of stability* [J. Cohen et al., 2021, abstract]; that is A3.

Example (`gd-by-hand`): GD on $L(\theta) = \frac{\lambda}{2}\theta^2$ with $\eta\lambda = 1.5$

Take $\lambda = 1$, $\eta = 1.5$, $\theta_0 = 1$. Then $\nabla L(\theta) = \theta$, so $\theta_{k+1} = \theta_k - 1.5\theta_k = -0.5\theta_k$, and $\theta_0 = 1$, $\theta_1 = -0.5$, $\theta_2 = 0.25$, $\theta_3 = -0.125$, $\theta_4 = 0.0625$. The iterate overshoots the minimum every step and lands on the other side, but each overshoot is half the last, so it converges while oscillating. (The test `test_gd_worked_example` checks exactly this sequence.)

A real symmetric matrix H has an *eigendecomposition* $H = Q\Lambda Q^\top$: Q is orthogonal (its columns q_i are orthonormal eigenvectors), $\Lambda = \mathrm{diag}(\lambda_1, ..., \lambda_d)$ holds the real eigenvalues. In the coordinates $z = Q^\top\theta$ the matrix becomes diagonal.

Problem (`gd-stability`): Stability of GD on a quadratic [Derive + Implement + Check, 35 min] **`[you]`**

(a) **`[Derive]`** For $L(\theta) = \frac{\lambda}{2}\theta^2$ with $\lambda > 0$, write θ_k in closed form. **Deliverable**: the ranges of $\eta\lambda$ for which GD (i) converges monotonically, (ii) converges while flipping sign, (iii) neither converges nor diverges, (iv) diverges.

(b) **`[Derive]`** For $L(\theta) = \frac{1}{2}\theta^\top H\theta$ with H symmetric positive definite, use $z = Q^\top\theta$ to show that each coordinate z_i follows (a) with $\lambda = \lambda_i$. Define the *contraction factor* $\rho(\eta)$ as the factor by which $\|\theta_k\|$ shrinks per step for large k and a generic start. **Deliverable**: (i) the condition on η and $\lambda_{\max}$ for convergence; (ii) $\rho(\eta)$ in terms of the eigenvalues; (iii) for eigenvalues $\{1, 10\}$: the critical step size η_c, the step size η^* that minimizes ρ, and $\rho(\eta^*)$, to three decimals.

(c) **`[Implement]`** Implement GD Equation 2 in `prereqs/gd.py`:

```
def gd(grad_fn, theta0, eta: float, steps: int) -> np.ndarray
```

- **`grad_fn, theta0`** maps θ of shape `(d,)` to $\nabla L(\theta)$; the start θ_0 (not modified).
- **`returns`** float64 array of shape `(steps+1, d)`, row k is θ_k; no early stopping or clipping.

To test your implementation, run `uv run pytest a0-prereqs -k test_gd` (the adapter `run_gd` in `tests/adapters.py` is pre-wired to this signature; edit it only if yours differs).

(d) **`[Check]`** Run `scripts/check_gd_stability.py`: GD on a 2-D quadratic with eigenvalues 1 and 10 in a rotated basis, η swept on a grid of spacing 0.0005; it measures ρ per η and prints the measured η_c, η^*, $\rho(\eta^*)$ (`gd_stability.png`). They should equal your (b)(iii) to the grid spacing. If your η^* is off, check which eigenvalue limits the rate on each side of it.

4 Fixed points, linearization and phase portraits

A *fixed point* of $\dot{\theta} = F(\theta)$ is a θ^* with $F(\theta^*) = 0$; started there, the system stays. Near it, write $\theta = \theta^* + \delta$ and Taylor-expand: $\dot{\delta} \approx J\delta$ with the *Jacobian* $J_{ij} = \partial F_i / \partial \theta_j(\theta^*)$. This linear ODE is solved by $\delta(t) = \sum_i c_i e^{\mu_i t} v_i$ for eigenpairs (μ_i, v_i) of J, so the eigenvalues decide what happens nearby (Table 1). A fixed point is *hyperbolic* if no eigenvalue has zero real part; for hyperbolic points the linearization shows the true local picture (the Hartman–Grobman theorem). A *phase portrait* draws the trajectories of a 2-D system in the plane. A *conserved quantity* is a function $C(\theta)$ constant along every trajectory, so trajectories lie on its level sets. For gradient flow $F = -\nabla L$, so $J = -\nabla^2 L$ is symmetric and its eigenvalues are real.

Eigenvalues μ_1, μ_2 of J	**Name**	**Nearby trajectories**
real, both < 0	stable node	all approach θ^*
real, both > 0	unstable node	all leave θ^*
real, opposite signs	saddle	approach along one direction, leave along the other
complex, Re < 0 / Re > 0	stable / unstable spiral	rotate inward / outward
purely imaginary	center	closed orbits (for the linear system)
at least one $= 0$	degenerate	linearization inconclusive in that direction

Table 1: Classification of a fixed point of a 2-D system by the eigenvalues of its Jacobian.

Example (`classify-by-hand`): Two Jacobians

For a 2×2 matrix the eigenvalues solve $\mu^2 - \tau\mu + \Delta = 0$ with trace τ and determinant Δ. (i) $J = \begin{pmatrix} 1 & 2 \\ 3 & 0 \end{pmatrix}$: $\tau = 1$, $\Delta = -6 < 0$, so the eigenvalues have opposite signs (3 and -2): a saddle. (ii) $J = \begin{pmatrix} -2 & 1 \\ -1 & -2 \end{pmatrix}$: $\tau = -4$, $\Delta = 5$, discriminant $\tau^2 - 4\Delta = -4 < 0$, so $\mu = -2 \pm i$: a stable spiral.

Problem (`phase-portrait`): The anchor model's phase portrait [Derive + Implement + Check, 45 min] **`[you]`**

(a) **`[Derive]`** Write the gradient flow of Equation 1 as two equations for $\dot{a}, \dot{b}$ and find every fixed point. **Deliverable**: the set of fixed points, and the loss at each.

(b) **`[Derive]`** Compute the Jacobian of the flow at a general (a, b), then its eigenvalues and eigenvectors at the origin and at a point $(a, 1/a)$ of the curve $ab = 1$. Classify each with Table 1 and say in words what a zero eigenvalue means here. **Deliverable**: the eigenvalues at the origin; the eigenvalues at $(a, 1/a)$ as functions of a; the direction of the zero-eigenvalue eigenvector relative to the curve $ab = 1$.

(c) **`[Derive]`** Verify by differentiating that $c = a^2 - b^2$ is conserved by the flow (Ang derives this from the rescaling symmetry; that argument is A1′s subject). Use it to find exactly which initializations end at the origin. **Deliverable**: that set of initializations, and the 1-D ODE the flow reduces to on it.

(d) **`[Implement]`** Implement Table 1 in `prereqs/fixed_points.py`:

`def classify_fixed_point(J, tol: float = 1e-9) -> str`

- **`J, tol`** the real 2×2 Jacobian; a real part, imaginary part or eigenvalue counts as zero if its size is at most `tol * max(1, max|J_ij|)`.
- **`returns`** one of `"stable node"`, `"unstable node"`, `"saddle"`, `"stable spiral"`, `"unstable spiral"`, `"center"`, `"degenerate"`.

To test your implementation, run `uv run pytest a0-prereqs -k test_classify_fixed_point` (the adapter `run_classify_fixed_point` in `tests/adapters.py` is pre-wired to this signature; edit it only if yours differs).

(e) **`[Check]`** Run `scripts/check_phase_portrait.py`. It prints the Jacobian's eigenvalues measured by finite differences at the origin and at four points of $ab = 1$, with your labels; compare with (b) to six digits. Its figure `phase_portrait.png` reproduces the post's phase portrait: trajectories from a 13×13 grid of initializations coloured by the branch of $ab = 1$ they reach, the dividing line in red, the conserved curves dashed. Expected: the branch is predicted by your (c) for 100% of the initializations, and c drifts by less than 10^{-6}.

5 Hessian, sharpness and trace

The *sharpness* of a loss at θ is $\lambda_{\max}(\nabla^2 L(\theta))$, the largest Hessian eigenvalue, i.e. the curvature along the steepest direction; the *trace* $\operatorname{tr} \nabla^2 L = \sum_i \lambda_i$ is the total curvature. A network with d parameters has a $d \times d$ Hessian, which nobody forms when d is large. What is cheap is a *Hessian-vector product* (HVP): since $\nabla^2 L(\theta) v = \nabla_\theta(\nabla L(\theta) \cdot v)$,

$$\nabla^2 L(\theta) v = \nabla_\theta [\nabla_\theta L(\theta)^\top v], \tag{4}$$

an HVP costs about two gradient evaluations by backpropagating twice (*double backprop*). Two algorithms turn HVPs into spectral information. *Power iteration* repeats

$$v_{k+1} = \frac{H v_k}{\|H v_k\|}, \quad \hat{\lambda}_k = v_k^\top H v_k, \tag{5}$$

which converges to the eigenvector of the largest-*magnitude* eigenvalue when it is unique (here always positive), with error shrinking like $|\lambda_2/\lambda_1|^k$. *Hutchinson's estimator* uses random probes z with $\mathbb{E}[zz^\top] = I$:

$$\operatorname{tr} H = \mathbb{E}[z^\top H z] \approx \frac{1}{S} \sum_{s=1}^{S} z_s^\top H z_s, \tag{6}$$

here with *Rademacher* probes, whose entries are independent and equal to ± 1 with probability $1/2$ each.

Problem (`hessian`): Sharpness and trace from Hessian-vector products [Implement + Derive + Check, 40 min] **`[you]`**

(a) **`[Implement]`** Implement in `prereqs/hessian.py`, with the parameters as one flat float64 tensor `params` of shape `(d,)` and `loss_fn(params)` a scalar tensor:

`def hvp(loss_fn, params, v) -> torch.Tensor`

- **`returns`** $\nabla^2 L(\text{params})\, v$ by Equation 4, detached, shape `(d,)`, without forming the Hessian.

`def power_iteration(hvp_fn, dim, iters, seed=0) -> tuple[float, torch.Tensor]`

- **returns** $\left(\hat{\lambda}, v\right)$ after `iters` steps of Equation 5 from a random unit v_0 drawn with `torch.Generator().manual_seed(seed)`; v a unit vector.

`def hutchinson_trace(hvp_fn, dim, samples, seed=0) -> float`

- **returns** the estimate Equation 6 with `samples` Rademacher probes drawn from a seeded generator.

To test your implementation, run `uv run pytest a0-prereqs -k test_hvp` (the adapter `run_hvp` in `tests/adapters.py` is pre-wired to this signature; edit it only if yours differs). Likewise `-k test_power_iteration` and `-k test_hutchinson_trace`; the references are the explicit anchor Hessian and `torch.autograd.functional.hessian` on a 193-parameter MLP. *Hint:* `torch.autograd.grad(loss, p, create_graph=True)` returns a gradient you can differentiate again, here as the scalar `g @ v`; make `p` a fresh leaf with `params.detach().requires_grad_(True)`.

(b) **[Derive]** From your Problem (`phase-portrait`) (b), write the sharpness and the trace of $\nabla^2 L$ at a minimum $(a, 1/a)$ of the anchor. Using Problem (`gd-stability`), find the interval of $a > 0$ in which the minimum $(a, 1/a)$ is linearly stable for GD with $\eta = 0.5$, and the same for $\eta = 1$. **Deliverable**: the two expressions; the two intervals (one of them is degenerate).

(c) **[Check]** Run `scripts/check_sharpness.py`. With your three functions it estimates sharpness and trace at 41 points of $ab = 1$ and plots them against a with the thresholds $2/\eta$ for $\eta = 0.5, 0.9, 1$ (`sharpness.png`); expected: sharpness within 10^{-8} relative of your expression, trace within a few Hutchinson standard deviations. It also marks the two endpoints of Problem (`flow-limit`) (d). This picture is where A3 starts. **Deliverable**: two sentences in `notes.md`: what the two stars say about the minimum GD at $\eta = 0.9$ could and could not settle at.

6 Eigendecomposition and SVD: linear regression as independent modes

A linear model $\hat{y} = Wx$ with $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ is trained on

$$L(W) = \frac{1}{2}\mathbb{E}\|Wx - y\|^2, \quad \Sigma_{xx} = \mathbb{E}[xx^\top], \quad \Sigma_{yx} = \mathbb{E}[yx^\top], \tag{7}$$

where the expectation is over the training set (an average over examples). Σ_{xx} is the input covariance (assume it is positive definite) and Σ_{yx} the input–output correlation. Every real matrix has a *singular value decomposition* (SVD) $M = USV^\top$, U, V orthogonal, S non-negative diagonal; for symmetric positive semidefinite M it is the eigendecomposition.

Problem (`modes`): Gradient flow of linear regression, mode by mode [Derive + Implement + Check, 30 min] **[you]**

(a) **[Derive]** Show that gradient flow on Equation 7 is

$$\dot{W} = \Sigma_{yx} - W\Sigma_{xx}. \tag{8}$$

Deliverable: $\nabla_W L$ and the minimizer $W^\star$.

(b) **[Derive]** Write $\Sigma_{xx} = U\Lambda U^\top$ and rotate, $\widetilde{W} = WU$. Show that each column of $\widetilde{W} - W^\star U$ (a *mode*) evolves on its own, and solve Equation 8. **Deliverable**: $W(t)$ in closed form in terms of W_0, $W^\star$, U, Λ and t; the time for the error in mode i to shrink by a factor e.

(c) **[Implement]** Type your closed form into `prereqs/modes.py` (no numerical integration):

`def mode_dynamics(W0, Sxx, Syx, t: float) -> np.ndarray`

- **returns** $W(t)$, shape `(d_out, d_in)`, solving Equation 8 with $W(0) =$ `W0`.

To test your implementation, run `uv run pytest a0-prereqs -k test_mode_dynamics` (the adapter `run_mode_dynamics` in `tests/adapters.py` is pre-wired to this signature; edit it only if yours differs).

(d) **[Check]** Run `scripts/check_modes.py`: a regression with $d_{\text{in}} = 5$, $d_{\text{out}} = 3$ and eigenvalues of Σ_{xx} spread over two decades (0.1 to 10). It plots the per-mode error $\|(W(t) - W^\star)u_i\|$ on a log scale (`modes.png`), fits each mode's decay rate, and compares your closed form with RK4 integration. Expected: straight lines whose fitted rates match your (b) to 10^{-3}, and closed form within 10^{-6} of RK4. The slowest mode sets how long training takes.

(e) **[Derive]** Suppose the inputs are whitened ($\Sigma_{xx} = I$) and $W_0 = 0$. Describe $W(t)$ through the SVD of Σ_{yx}: what are its singular vectors and how do its singular values grow? **Deliverable**: one sentence.

Note: This is the one-layer case of Saxe, McClelland and Ganguli's analysis [A. M. Saxe et al., 2014, §1, pp. 2–4]. With two layers, $\hat{y} = W_2 W_1 x$, the modes become the singular modes of Σ_{yx} and each is learned on an S-shaped curve whose timing depends on its singular value; A1 derives it. Read §1 of the paper for the general case.

7 SDE intuition: drift plus noise, the Ornstein–Uhlenbeck process

SGD is GD plus noise; its continuous-time language is the *stochastic differential equation* (SDE). *Brownian motion* W_t is the random path with $W_0 = 0$, continuous paths, and independent Gaussian increments $W_{t+h} - W_t \sim \mathcal{N}(0, h)$: the variance of an increment equals its duration, so its typical size is $\sqrt{h}$, not h. An *Itô SDE*

$$dX = \mu(X)\, dt + \sigma\, dW \tag{9}$$

reads as "drift μ plus noise of scale σ", and operationally it is the $\Delta t \to 0$ limit of the *Euler–Maruyama* scheme

$$X_{k+1} = X_k + \mu(X_k)\Delta t + \sigma\sqrt{\Delta t}\,\xi_k, \quad \xi_k \sim \mathcal{N}(0, 1) \text{ independent.} \tag{10}$$

The *Ornstein–Uhlenbeck* (OU) process takes $\mu(X) = -\theta X$ with $\theta > 0$: a pull toward 0 of strength θ (here θ is a rate, not a parameter vector) against constant random kicks. It is the local picture of SGD near a minimum, which part (f) makes precise.

Problem (ou): The Ornstein–Uhlenbeck process [Derive + Implement + Check + Predict, 35 min] **[you]**

(a) **[Derive]** For $dX = -\theta X\, dt + \sigma\, dW$, $X_0 = x_0$, take expectations in Equation 10, let $\Delta t \to 0$, and solve. **Deliverable**: the mean $m(t) = \mathbb{E}[X_t]$.

(b) **[Derive]** Do the same for the variance: from Equation 10 get a recursion $v_{k+1} = (\ldots)v_k + (\ldots)$, keep terms of first order in Δt, and solve the resulting ODE. **Deliverable**: $v(t) = \text{Var}[X_t]$ and the stationary variance $v_\infty = \lim_{t\to\infty} v(t)$.

(c) **[Implement]** Implement Equation 10 for OU in `prereqs/sde.py`:

`def ou_simulate(x0, theta, sigma, dt, steps, n_paths, rng) -> np.ndarray`

- **returns** float64 array of shape `(steps+1, n_paths)`, row k the paths at time $k\Delta t$, all randomness from the numpy Generator `rng`.

To test your implementation, run `uv run pytest a0-prereqs -k test_ou_simulate` (the adapter `run_ou_simulate` in `tests/adapters.py` is pre-wired to this signature; edit it only if yours differs).

(d) **[Check]** Type your (a) and (b) into the `FORMULAS` block at the top of `scripts/check_ou.py` and run it: 5000 paths with $\theta = 1$, $\sigma = 0.5$, $x_0 = 2$, $\Delta t = 0.01$. Expected: the sample mean and variance follow your curves and the histogram at $t = 8$ matches $\mathcal{N}(0, v_\infty)$, within the sampling error $v_\infty\sqrt{2/5000} \approx 0.0025$ of a variance estimate plus an $O(\Delta t)$ bias (`ou.png`).

(e) **[Predict]** The script then runs Euler–Maruyama with large steps, $\theta\Delta t$ up to 1.9. First write in `notes.md` what you expect its stationary variance to do as $\theta\Delta t$ grows toward 2. Then solve the *exact* recursion of (b) for its fixed point (no first-order truncation) and type it into `em_stationary_var_formula`. **Deliverable**: the prediction; the exact discrete-time stationary variance.

(f) **[Derive]** Near a minimum θ^* of a 1-D loss with curvature λ, suppose SGD's gradient is $\lambda(\theta - \theta^*) + \xi$ with zero-mean noise of variance s^2. Show that SGD with step η is exactly Equation 10 for an OU process, and identify its θ, σ and Δt. In one `notes.md` paragraph: how far do SGD's iterates spread around the minimum, as a function of η, λ, s, and where does (e) meet Problem (`gd-stability`)? **Deliverable**: the identification and the paragraph.

Note: On a manifold of minima the same balance (noise pushes off, curvature pulls back) also drives a slow drift *along* it toward flatter points, shown for label-noise SGD by Blanc et al. [G. Blanc et al., 2020, §1]; A4 derives it on the anchor.

8 Framing: what a theory of training dynamics is for

Problem (`framing`): Read the two framing papers [Read, 30 min] **[you]**

Read Simon et al. [J. Simon et al., 2026] §1 (pp. 2–5), Table 1 and the opening of §2.1 (pp. 6–7): the five bodies of work (solvable settings, insightful limits, simple empirical laws, theories of hyperparameters, universal phenomena). Then read the abstract and §2.4 "Predict, Intervene, Design" (pp. 4–5) of Biderman et al. [S. Biderman et al., 2026], whose three capabilities form a hierarchy, each needing everything the one before it does. Both PDFs are in `papers/`. Answer briefly in `notes.md`:

(a) Which of the five bodies of work does the anchor model belong to, and why? Name a second one the course reaches from it.

(b) Biderman et al. want to "predict outcomes from early training signals". What would that mean for the anchor model: which outcome, from which signal, at what time?

(c) One claim in either paper you do not believe yet, and what evidence would change your mind.

9 What to write down

In `notes.md`, three to five sentences: when GD is well approximated by gradient flow and what breaks first when it is not; what the eigenvalues of a Jacobian, a Hessian and a covariance each told you today and how those three are related for gradient flow; why the noise in an SDE scales as $\sqrt{\Delta t}$; and one question you would bring to a tutorial.

References

[1] J. M. L. Ang, "Training Dynamics." 2026. Science of Deep Learning: Taster Series, 16 October 2026. Offline: papers/ang2026-training-dynamics-post.md.

[2] J. Cohen *et al.*, "Gradient Descent on Neural Networks Typically Occurs at the Edge of Stability." International Conference on Learning Representations (ICLR), 2021. arXiv:2103.00065. Offline: papers/cohen2021-edge-of-stability.pdf.

[3] A. M. Saxe *et al.*, "Exact Solutions to the Nonlinear Dynamics of Learning in Deep Linear Neural Networks." International Conference on Learning Representations (ICLR), 2014. arXiv:1312.6120. Offline: papers/saxe2014-exact-solutions-deep-linear.pdf.

[4] G. Blanc *et al.*, "Implicit Regularization for Deep Neural Networks Driven by an Ornstein-Uhlenbeck Like Process." Conference on Learning Theory (COLT), PMLR 125, 2020. arXiv:1904.09080. Offline: papers/blanc2020-ou-implicit-regularization.pdf.

[5] J. Simon *et al.*, "There Will Be a Scientific Theory of Deep Learning." 2026. arXiv:2604.21691. Offline: papers/simon2026-scientific-theory-deep-learning.pdf.

[6] S. Biderman *et al.*, "Position: Don't Just "Fix it in Post": A Science of AI Must Study Training Dynamics." Proceedings of the 43rd International Conference on Machine Learning (ICML), PMLR 306, 2026. arXiv:2606.06533. Offline: papers/biderman2026-dont-fix-it-in-post.pdf.